

Instituto Tecnológico de Puerto Rico

Recinto de San Juan

LABORATORIO - SEMANA 2

Mejora y control de versiones de un programa en Python con Git y GitHub

Nombre del estudiante: _Justin k. Fernandez Ramos	Fecha: 24/08/26
Sección: _____	URL del repositorio: _____

Curso	TESI 2105 - Programación Orientada a Objetos
Semana	2
Laboratorio	Laboratorio 1
Tema integrado	Elementos fundamentales del programa + control básico de versiones
Objetivos del temario	G2, G3, C1 y E1
Duración sugerida	3 horas
Punto de partida	Repositorio de GitHub creado previamente con un programa .py
Evidencia	Programa actualizado + historial Git + enlace al repositorio + capturas solicitadas

PROPÓSITO

Aplicar los elementos fundamentales de programación estudiados en la Semana 2 mediante la mejora de un programa de Python desarrollado previamente. El estudiante utilizará Git y GitHub para identificar, registrar y publicar los cambios realizados, siguiendo un flujo básico de control de versiones.

PUNTO DE PARTIDA: Este laboratorio parte de un repositorio de GitHub que ya contiene un programa .py del estudiante. No debes crear un repositorio nuevo ni sustituir tu programa por el ejemplo de otro estudiante.

RESULTADOS DE APRENDIZAJE

- Ejecutar y verificar el funcionamiento de un programa de Python previamente desarrollado.
- Modificar el programa incorporando variables, entrada de datos, conversión, operadores, procesamiento y salida.
- Utilizar Git para identificar archivos modificados y revisar diferencias entre versiones.
- Preparar y registrar cambios mediante git add y git commit.
- Publicar el nuevo commit en GitHub mediante git push y verificar el historial del repositorio.

REQUISITOS ANTES DE COMENZAR

- Cuenta activa de GitHub y sesión iniciada en el navegador.
- Repositorio personal creado previamente para el curso.
- Archivo de Python (.py) dentro del repositorio.
- El programa debe ejecutarse antes de iniciar las modificaciones del laboratorio.
- Conexión a Internet estable.

FLUJO GENERAL DEL LABORATORIO

1. Abrir repositorio	2. Crear Codespace	3. Ejecutar programa	4. Modificar código
5. status / diff	6. add / commit	7. log / push	8. Verificar en GitHub

PARTE A - ABRIR EL REPOSITORIO Y CREAR EL CODESPACE

1. Accede a <https://github.com> e inicia sesión con la misma cuenta que utilizaste para crear tu repositorio.
2. Abre el repositorio que contiene tu programa de Python.
3. Verifica que puedes observar tu archivo con extensión .py antes de continuar.
4. Presiona el botón Code.
5. Selecciona la pestaña Codespaces.
6. Presiona Create codespace on main y espera a que Visual Studio Code termine de cargar en el navegador.

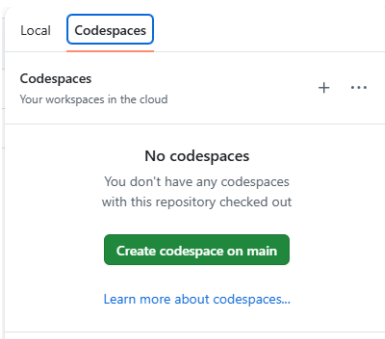


Figura 1. Opción para crear un Codespace desde la rama main.

SI NO APARECE CODESPACES: Si al presionar Code solo observas opciones como HTTPS, GitHub CLI, GitHub Desktop o Download ZIP, verifica que hayas iniciado sesión. Después de iniciar sesión, vuelve a abrir Code y selecciona Codespaces.

CHECKPOINT 1

- ✓ El repositorio correcto está abierto.
- ✓ El Codespace se creó desde la rama main.
- ✓ En el panel Explorer aparece tu archivo .py.

PARTE B - VERIFICAR EL ENTORNO DE TRABAJO

Cuando Codespaces abra, identifica tres áreas: Explorer a la izquierda, el editor en el centro y la Terminal en la parte inferior. Si la terminal no aparece, selecciona Terminal > New Terminal.

En la terminal, ejecuta los siguientes comandos uno por uno:

```
git --version  
pwd  
git status
```

El comando git status debe reconocer el repositorio. Un resultado típico puede indicar que estás en la rama main y que no hay cambios pendientes.

```
On branch main  
Your branch is up to date with 'origin/main'.  
  
nothing to commit, working tree clean
```

NOTA: Si git commit luego solicita tu identidad, configura tu nombre y correo con los comandos indicados en la sección Solución de problemas. No es necesario configurarlos si Git ya los reconoce.

CHECKPOINT 2

- ✓ git --version muestra una versión instalada.
- ✓ pwd muestra la carpeta del repositorio del Codespace.
- ✓ git status funciona sin el mensaje “not a git repository”.

PARTE C - EJECUTAR EL PROGRAMA ORIGINAL

7. Localiza tu archivo .py en Explorer.
8. Ejecuta el programa desde la terminal utilizando python seguido del nombre real del archivo.

```
python nombre_archivo.py
```

Ejemplo:

```
python programa_tesi2105.py
```

9. Observa la salida y verifica que el programa funcione antes de modificarlo.

NO CONTINÚES SI EL PROGRAMA NO FUNCIONA: Si el programa presenta un error antes de modificarlo, documenta el error y consulta al profesor. El laboratorio requiere una versión inicial funcional para poder comparar cambios.

CHECKPOINT 3

- ✓ El programa original se ejecuta.
- ✓ Puedes identificar qué datos recibe, qué procesa y qué resultado muestra.

PARTE D - MEJORAR EL PROGRAMA

Realiza una mejora pequeña y clara en tu programa. Debes conservar el propósito original del código y añadir o verificar los siguientes elementos de la Semana 2:

- Al menos dos variables con nombres descriptivos.
- Una entrada de texto mediante `input()`.
- Al menos una conversión de datos utilizando `int()` o `float()`.
- Al menos una operación matemática con operadores aritméticos.
- Un proceso que utilice los datos recibidos para producir un resultado.
- Una salida clara utilizando `print()`.
- Una constante escrita en mayúsculas, cuando aplique al propósito de tu programa.

Ejemplo de referencia (no lo copies como sustitución de tu programa):

```
CARGO_SERVICIO = 15.00
nombre = input("Nombre del cliente: ")
horas = float(input("Horas trabajadas: "))
tarifa = float(input("Tarifa por hora: "))
total = (horas * tarifa) + CARGO_SERVICIO
print(f"Cliente: {nombre}")
print(f"Total estimado: ${total:.2f}")
```

10. Guarda el archivo con `Ctrl + S`.

11. Ejecuta nuevamente el programa para confirmar que la modificación funciona.

CHECKPOINT 4

- ✓ El archivo fue guardado.
- ✓ El programa modificado se ejecuta sin errores.
- ✓ La mejora utiliza los elementos solicitados.

PARTE E - OBSERVAR LOS CAMBIOS CON GIT

Ahora utilizarás Git para comprobar que el archivo cambió respecto a la versión almacenada en el repositorio.

12. Ejecuta `git status`.

```
git status
```

Debes observar tu archivo `.py` como modificado, generalmente bajo `Changes not staged for commit`.

13. Ejecuta git diff para ver las líneas que cambiaron.

```
git diff
```

Git suele representar las líneas eliminadas con - y las líneas añadidas con +. Si la pantalla queda en modo de visualización, presiona q para regresar a la terminal.

CHECKPOINT 5

- ✓ git status identifica el archivo modificado.
- ✓ git diff muestra las líneas que añadiste o cambiaste.

PARTE F - PREPARAR Y REGISTRAR EL CAMBIO

Git utiliza dos pasos distintos antes de publicar una nueva versión: primero prepara el archivo y luego registra el cambio.

14. Prepara únicamente tu archivo de Python con git add.

```
git add nombre_archivo.py
```

Ejemplo:

```
git add programa_tesi2105.py
```

15. Verifica nuevamente el estado.

```
git status
```

El archivo debe aparecer ahora bajo Changes to be committed.

16. Crea un commit con un mensaje breve que describa la mejora.

```
git commit -m "Mejora entrada y cálculo del programa"
```

MENSAJE DEL COMMIT: Utiliza mensajes descriptivos. Evita mensajes como “cambio”, “trabajo”, “prueba” o “final”. Un mensaje útil explica qué se modificó.

17. Verifica que el directorio quede limpio.

```
git status
```

CHECKPOINT 6

- ✓ El archivo fue preparado con git add.
- ✓ El commit se creó correctamente.
- ✓ git status indica que no quedan cambios pendientes.

PARTE G - VER EL HISTORIAL Y PUBLICAR EN GITHUB

18. Consulta el historial resumido de commits.

```
git log --oneline
```

Tu commit nuevo debe aparecer en la primera línea. Si la terminal entra en modo de visualización, presiona q para salir.

19. Envía el nuevo commit al repositorio remoto.

```
git push
```

20. Regresa a la pestaña del navegador donde está GitHub y actualiza la página del repositorio.

21. Verifica que el mensaje del nuevo commit aparezca y abre el archivo .py para confirmar que contiene la modificación.

CHECKPOINT 7

- ✓ git log --oneline muestra el nuevo commit.
- ✓ git push termina sin errores.
- ✓ GitHub muestra el nuevo mensaje de commit y el archivo actualizado.

PARTE H - EVIDENCIA A ENTREGAR

Entrega un solo documento de evidencia en PDF o Word con las siguientes capturas y el enlace al repositorio:

Evidencia	Nombre	Qué debe mostrar
1	Repositorio inicial	Captura del repositorio donde se vea el archivo .py antes o al comenzar la práctica.
2	Cambio detectado	Captura de la terminal mostrando git status después de modificar el programa.
3	Commit registrado	Captura de git log --oneline donde aparezca el nuevo commit.
4	Repositorio actualizado	Captura del repositorio en GitHub después de git push mostrando el mensaje del commit.
5	Enlace	URL funcional del repositorio de GitHub utilizado en el laboratorio.

PREGUNTAS DE REFLEXIÓN

1. ¿Qué información te ofrece git status antes y después de modificar un archivo?

Antes de modificarlo muestra que el directorio de trabajo está limpio.

2. ¿Qué diferencia observaste entre git add y git commit?

añade los cambios al área de preparación staggin area para seleccionarlos, mientras que git commit guarda esos cambios de forma permanente en el historial del repositorio local con un mensaje descriptivo.

3. ¿Qué información adicional pudiste ver con git diff?

Muestra el detalle exacto de las líneas agregadas en verde con un + y eliminadas en rojo con un -dentro del archivo en comparación con la última versión guardada.

4. ¿Para qué sirve git push dentro del flujo de trabajo realizado?

Sirve para subir y sincronizar los confirmaciones commits del repositorio local hacia el repositorio remoto en la nube como GitHub o GitLab.

5. ¿Qué ventaja tiene conservar un historial de versiones cuando continuas desarrollando un programa?

Permite rastrear quién hizo cada cambio y por qué, recuperar versiones anteriores si ocurre un error grave, y trabajar en nuevas funciones sin alterar el código que ya funciona.

SOLUCIÓN DE PROBLEMAS

Situación	Qué hacer
No aparece Codespaces	Confirma que iniciaste sesión en GitHub. Abre Code nuevamente y selecciona la pestaña Codespaces.
git status dice “not a git repository”	Verifica que el Codespace fue creado directamente desde tu repositorio. Ejecuta pwd y confirma que estás en la carpeta del proyecto.
El commit solicita nombre o correo	Configura tu identidad y vuelve a ejecutar el commit.
git log ocupa toda la terminal	Presiona q para regresar a la línea de comandos.
git push muestra un error de permisos	Confirma que el repositorio pertenece a tu cuenta o que tu cuenta tiene permisos de escritura y que el Codespace se abrió con la cuenta correcta.
Git no detecta el cambio	Guarda el archivo con Ctrl + S y vuelve a ejecutar git status.

Solo si Git solicita tu identidad, utiliza los siguientes comandos reemplazando los datos por los tuyos:

```
git config --global user.name "Nombre Apellido"
git config --global user.email "tu_correo@ejemplo.com"
```

RESUMEN DE COMANDOS

Comando	Función
git --version	Verifica que Git esté disponible.
pwd	Muestra la carpeta actual.
git status	Muestra el estado de los archivos y del repositorio.
git diff	Muestra diferencias entre la versión guardada y los cambios actuales.
git add archivo.py	Prepara el archivo para el próximo commit.
git commit -m "mensaje"	Registra una nueva versión local con un mensaje.
git log --oneline	Muestra el historial resumido de commits.
git push	Envía los commits locales al repositorio de GitHub.

RÚBRICA DE EVALUACIÓN - 100 PUNTOS

Criterio	Excelente	Adecuado	En desarrollo	Puntos
Acceso y preparación del entorno	Accede al repositorio correcto, crea el Codespace y verifica Git sin asistencia.	Completa el acceso con una corrección menor.	Requiere asistencia frecuente o trabaja en un repositorio incorrecto.	10
Ejecución del programa original	Ejecuta el programa existente y verifica su funcionamiento antes de modificarlo.	Ejecuta el programa con una corrección menor.	No logra demostrar el funcionamiento inicial.	10
Mejora del programa en Python	Integra claramente variables, entrada, conversión, operador(es), procesamiento y salida; conserva el propósito del programa.	Integra la mayoría de los elementos con errores menores.	Faltan varios elementos o el programa no funciona correctamente.	25
Uso de git status y git diff	Utiliza ambos comandos en el momento correcto e interpreta los cambios observados.	Utiliza los comandos con orientación menor.	No evidencia uno o ambos comandos o no identifica los cambios.	15
Registro de cambios: add y commit	Prepara el archivo correcto y crea un commit con mensaje descriptivo.	Completa el proceso con un error menor o mensaje poco descriptivo.	No completa correctamente add/commit.	20
Publicación y verificación: push/log	Envía los cambios a GitHub, verifica el historial y confirma el cambio en el repositorio.	Completa el proceso con orientación menor.	No logra publicar o verificar el cambio.	10
Evidencias y reflexión	Entrega todas las capturas, enlace funcional y respuestas completas.	Falta una evidencia o respuesta menor.	Faltan varias evidencias o respuestas.	10

Puntuación obtenida: _____ / 100

CIERRE DEL LABORATORIO

Al finalizar habrás realizado un flujo completo de control de versiones sobre un programa real: abriste tu repositorio, ejecutaste el código, realizaste una mejora, verificaste las diferencias, registraste una nueva versión y la publicaste en GitHub.

IDEA CLAVE: El objetivo no es memorizar todos los comandos. Debes comprender qué ocurre con el programa en cada etapa: modificar → revisar → preparar → registrar → publicar.